UNIVERSIDAD MARIANO GÁLVEZ DE GUATEMALA

Facultad de Ingeniería en Sistemas de Información y Ciencias de la Computación

Análisis de Sistemas II — Código 037



PROYECTO FINAL

Control de inventario y préstamos de laboratorio

Automatización con n8n, PostgreSQL y Docker en un entorno local

Documento técnico

Autora: Irma Yaritza Katiuxca Jagan Xol

Carné: 4090-23-9748

Docente: Ing. Edgar Raúl Molina Rey

Sección: Unica

Modalidad: Individual

Cobán, Alta Verapaz — 26 de Octubre de 2026

Índice

1. Introducción 4
2. Planteamiento del problema 4
2.1 Situación actual 4
2.2 Problema identificado 5
2.3 Objetivo general 5
2.4 Objetivos específicos 5
2.5 Alcance y limitaciones 6
3. Arquitectura local 6
3.1 Componentes 6
3.2 Decisiones de diseño 7
4. Modelo de datos 8
4.1 Reglas de integridad 8
5. Descripción de los workflows 9
5.1 WF1 – Ingesta de préstamos 11
5.1.1 Nodos 11
5.1.2 Reglas de validación 11
5.2 WF2 – Procesamiento de préstamos 12
5.2.1 Nodos 13
5.3 WF3 – Salida de reportes 14
5.3.1 Nodos 14
5.4 WF4 – Manejo de errores 15
5.5 Resumen de nodos utilizados 16
6. Cómo trabajan juntos 17
7. Manejo de errores y de los datos 17
7.1 Errores esperados 17
7.2 Errores inesperados 17
7.3 Consistencia de la información 18
8. Consideraciones de seguridad 18
9. Pruebas realizadas 20
10. Instalación y reproducibilidad 22
11. Aplicación web de préstamos 22
11.1 Tecnologías y arquitectura 23
11.2 Flujo de una solicitud 23
11.3 Seguridad de la aplicación 25

12. Pruebas unitarias automatizadas 28
12.1 Herramientas 29
12.2 Casos probados 30
12.3 Cobertura de código 30
13. Modelado UML 32
13.1 Diagrama de casos de uso 32
13.2 Diagrama de actividad 33
13.3 Diagrama de secuencia 34
13.4 Diagrama de clases 35
13.5 Diagrama de despliegue 36
14. Conclusiones 37
14.1 Trabajo futuro 38
15. Referencias y recursos de apoyo 38

1. Introducción

Este documento describe el análisis, el diseño y la implementación de un sistema de control de inventario y préstamos para un laboratorio, desarrollado como Proyecto Final del curso Análisis de Sistemas II. El sistema automatiza el proceso completo de un préstamo de equipo: la recepción de la solicitud, la validación de los datos, la verificación del stock disponible, el registro del préstamo, la actualización del inventario y la detección diaria de préstamos vencidos.

La solución se construyó con n8n como motor de automatización, PostgreSQL como base de datos y Docker como plataforma de despliegue, todo en un entorno 100 % local. Como interfaz para los usuarios se desarrolló además una aplicación web con Spring Boot y Thymeleaf, que registra el número de celular del estudiante, entrega un comprobante en PDF de cada préstamo y se ejecuta en su propio contenedor Docker, respaldada por 104 pruebas unitarias automatizadas con JUnit, Mockito y JaCoCo. La ciberseguridad se trató como un eje transversal del diseño: cada componente incorpora al menos una medida de protección sobre las credenciales o sobre los datos que maneja.

El código fuente, los workflows exportados, los casos de prueba y los diagramas están disponibles en el repositorio público:

https://github.com/kathyjaganx/proyecto-final-as2-inventario

2. Planteamiento del problema

2.1 Situación actual

En muchos laboratorios universitarios los préstamos de equipo (laptops, proyectores, multímetros, kits de electrónica, cables) se controlan en cuadernos o en hojas de cálculo que se llenan a mano. Este método depende por completo de la persona encargada: no valida que los datos estén completos, no impide prestar más unidades de las que existen y no avisa cuando un equipo no se devuelve a tiempo.

2.2 Problema identificado

El control manual produce cuatro problemas principales:

- **Inconsistencia del inventario:** el stock registrado no coincide con el real porque los préstamos y las devoluciones no siempre se anotan.
- **Datos incompletos o erróneos:** carnés mal escritos, nombres ilegibles o fechas de devolución imposibles.
- **Equipo no devuelto:** no hay un mecanismo que detecte a tiempo los préstamos vencidos.
- **Falta de trazabilidad:** no queda registro de quién solicitó qué, cuándo, ni por qué se rechazó una solicitud.

2.3 Objetivo general

Automatizar el proceso de préstamos de equipo de laboratorio mediante workflows de n8n en un entorno local con Docker y PostgreSQL, garantizando la validación de los datos, la consistencia del inventario, la trazabilidad de todas las operaciones y la protección de las credenciales y los datos personales de los estudiantes.

2.4 Objetivos específicos

1. Recibir las solicitudes de préstamo mediante un webhook protegido con una clave de acceso y validar sus datos antes de procesarlas.
2. Registrar el préstamo y descontar el stock en una sola operación atómica, de modo que el inventario nunca quede inconsistente.
3. Detectar diariamente los préstamos vencidos y generar un reporte verificable en la base de datos y en un archivo CSV.
4. Registrar toda operación en una bitácora de auditoría y todo fallo en una tabla de errores.
5. Ofrecer una aplicación web que envíe las solicitudes al sistema, registre el número de celular del estudiante y entregue un comprobante en PDF de cada préstamo, y verificar su funcionamiento con pruebas unitarias automatizadas que cubran al menos el 80 % del código.

2.5 Alcance y limitaciones

El sistema cubre la solicitud y el registro de préstamos, el control del stock, la detección de vencimientos, la auditoría y el manejo de errores. Quedan fuera del alcance de esta versión el registro de devoluciones (la base de datos ya incluye el campo fecha_devolucion_real y el estado DEVUELTO para incorporarlo después) y el envío de notificaciones por correo.

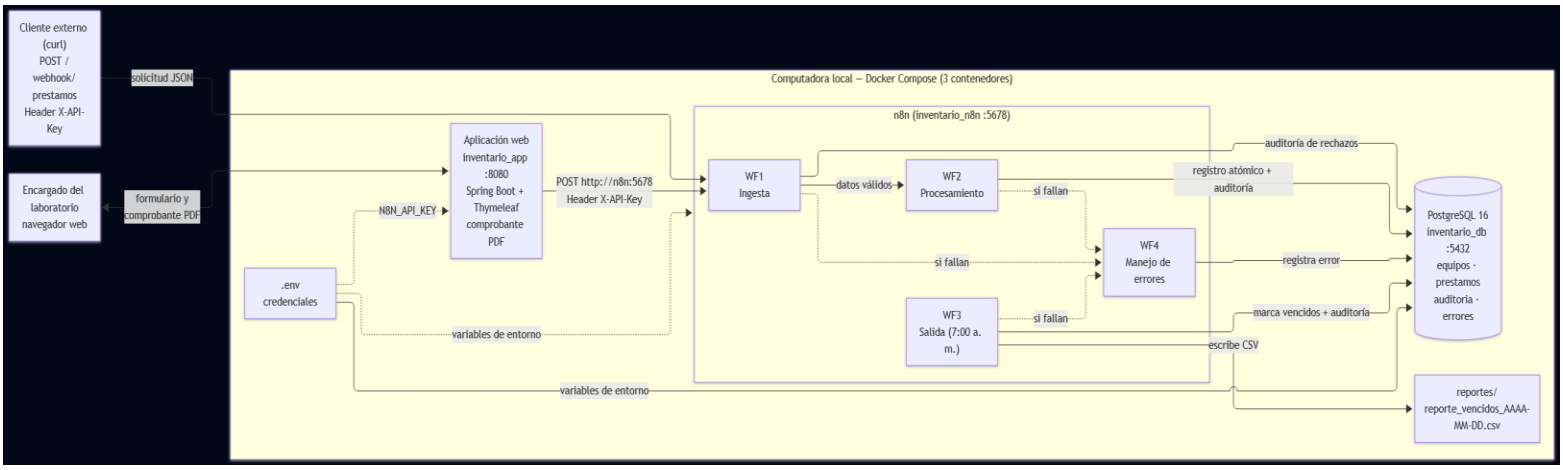
3. Arquitectura local

3.1 Componentes

La solución se despliega con Docker Compose en una sola computadora. El archivo docker-compose.yml define tres contenedores y dos volúmenes persistentes:

Componente	Tecnología	Función
inventario_app	Spring Boot 4.1 + Thymeleaf + Apache PDFBox (puerto 8080)	Aplicación web donde se registran los préstamos. Envía cada solicitud al webhook del WF1 con la clave X-API-Key y muestra el resultado. Genera el comprobante en PDF. Su imagen se construye con el Dockerfile del módulo app-prestamos y corre con un usuario sin privilegios.
inventario_n8n	n8n (puerto 5678)	Ejecuta los cuatro workflows. Guarda su configuración y sus credenciales cifradas en el volumen n8n_data.
inventario_db	PostgreSQL 16 (puerto 5432)	Almacena el inventario, los préstamos, la auditoría y los errores en el volumen postgres_data.
Carpeta reportes/	Volumen compartido	Recibe los archivos CSV que genera el WF3. Es la única carpeta a la que n8n tiene acceso.
Archivo .env	Variables de entorno	Contiene las credenciales. Docker Compose las inyecta en los contenedores y Git lo excluye del repositorio.

tenemos el diagrama de arquitectura del sistema



3.2 Decisiones de diseño

- **Docker Compose:** permite levantar todo el entorno con un solo comando y que cualquier persona lo reproduzca en otra computadora.
- **PostgreSQL en lugar de archivos:** ofrece transacciones y restricciones de integridad, indispensables para que el stock sea siempre correcto.
- **Separación en cuatro workflows:** cada workflow tiene una sola responsabilidad (ingesta, procesamiento, salida y errores), lo que facilita probarlos y mantenerlos por separado.
- **Comunicación interna por nombre de servicio:** n8n se conecta a la base de datos con el host postgres, dentro de la red privada de Docker.
- **Zona horaria America/Guatemala:** configurada en los tres contenedores para que las fechas de auditoría y los cálculos de vencimiento correspondan a la hora local.

- Aplicación web en contenedor: la aplicación también corre en Docker, arranca sola junto con los demás contenedores (restart: unless-stopped) y llama a n8n por su nombre de servicio (http://n8n:5678).

  Por lo tanto muestro el contenedor del proyecto de ejecución

```
PS C:\Users\KATY\Desktop\ProyectoAnalisis> docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED       STATUS                 PORTS                                              NAMES
a4f84969f37e   n8nio/n8n:latest   "tini -- /docker-ent…"   2 hours ago   Up 2 hours             0.0.0.0:5678->5678/tcp, [::]:5678->5678/tcp        inventario_n8n
28e8c6affb72   postgres:16        "docker-entrypoint.s…"   18 hours ago  Up 3 hours (healthy)   0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp        inventario_db
PS C:\Users\KATY\Desktop\ProyectoAnalisis>
```

4. Modelo de datos

El script db/init.sql crea las cuatro tablas al iniciar PostgreSQL por primera vez y carga cinco equipos de prueba.

Tabla	Campos principales	Propósito
equipos	id_equipo, codigo (único), nombre, categoria, stock_total, stock_disponible, activo	Inventario del laboratorio.
prestamos	id_prestamo, carne, nombre_estudiante, telefono, id_equipo (FK), cantidad, fecha_prestamo, fecha_devolucion_esperada, fecha_devolucion_real, estado	Registro de cada préstamo y su estado: ACTIVO, DEVUELTO o VENCIDO.
auditoria	id_auditoria, fecha, workflow, accion, resultado, detalle (JSONB)	Bitácora de ejecución de todas las operaciones, con resultado EXITO, RECHAZADO o ERROR.
errores	id_error, fecha, workflow, nodo, mensaje, id_ejecucion, datos (JSONB)	Fallos capturados por el WF4.

4.1 Reglas de integridad

La base de datos hace cumplir por sí misma las reglas del negocio, aunque un workflow tuviera un error:

- chk_stock_coherente: el stock disponible nunca puede ser mayor que el stock total ni menor que cero.
- chk_fechas: la fecha de devolución esperada no puede ser anterior a la fecha del préstamo.
- La cantidad prestada debe ser mayor que cero y el estado solo admite los valores ACTIVO, DEVUELTO o VENCIDO.

- La llave foránea id_equipo impide registrar préstamos de equipos inexistentes.

- El teléfono del estudiante debe tener exactamente 8 dígitos (restricción CHECK). No es único: un mismo estudiante puede tener varios préstamos con su número, y un índice sobre la columna permite consultarlos rápidamente.

- Los índices sobre estado y fecha_devolucion_esperada aceleran la búsqueda diaria de vencidos.

Los campos detalle y datos usan el tipo JSONB, que permite guardar información estructurada distinta en cada evento (los datos de la solicitud, la lista de errores o el reporte completo) sin cambiar el esquema.

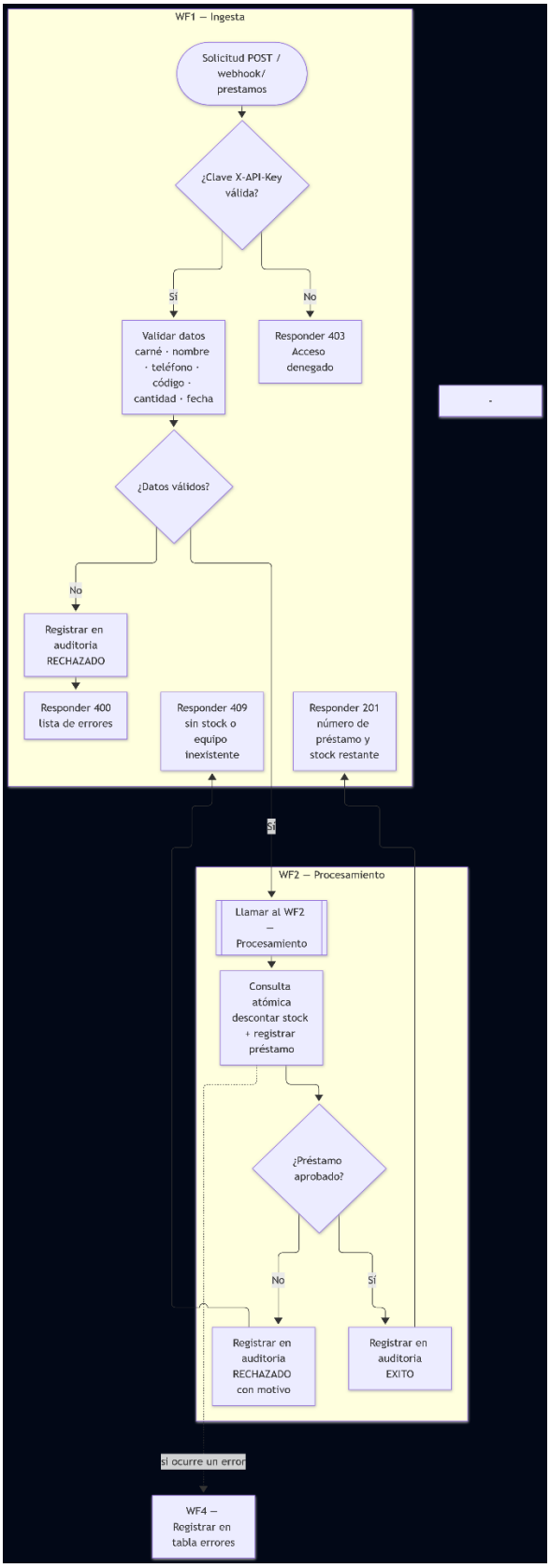
5. Descripción de los workflows

El sistema está formado por cuatro workflows que cumplen los tres tipos exigidos (ingesta, procesamiento y salida) más uno dedicado al manejo de errores.

Workflow	Tipo	Qué hace
WF1 – Ingesta de préstamos	Ingesta	Recibe la solicitud por un webhook protegido, valida los datos y decide si la rechaza o la envía a procesamiento.
WF2 – Procesamiento de préstamos	Procesamiento	Verifica el stock, registra el préstamo y descuenta el inventario en una sola operación.
WF3 – Salida de reportes	Salida	Cada día a las 7:00 a. m. marca los préstamos vencidos y genera un reporte en la base de datos y en un archivo CSV.
WF4 – Manejo de errores	Errores	Se activa cuando falla cualquier otro workflow y registra el error.

La solución automatiza el proceso completo con cuatro workflows de n8n que trabajan sobre una base de datos PostgreSQL, ambos ejecutándose en contenedores Docker:

Mostrando el diagrama del flujo principal que seria el WF1 Y EL WF2



5.1 WF1 – Ingesta de préstamos

Objetivo: ser la única puerta de entrada al sistema. Ninguna solicitud llega a la base de datos sin pasar antes por este workflow.

Disparador: nodo Webhook que recibe peticiones POST en /webhook/prestamos. Exige la cabecera X-API-Key (Header Auth): sin la clave correcta, n8n responde 403 y el workflow ni siquiera se ejecuta.

Es la puerta de entrada del sistema. Recibe la solicitud por HTTP, comprueba la clave de acceso, valida los datos y decide si la rechaza o la envía a procesamiento.

5.1.1 Nodos

Nodo	Tipo	Función
Webhook	Webhook	Recibe POST /webhook/prestamos. Exige la cabecera X-API-Key mediante Header Auth.
Validar datos	Code	Limpia y valida carné, nombre, teléfono, código de equipo, cantidad y fecha. Devuelve valido y la lista de errores.
¿Datos validos?	If	Separa el flujo en datos válidos e inválidos.
Auditoria rechazo	Postgres (Insert)	Registra la solicitud rechazada con sus errores.
Responder rechazo	Respond to Webhook	Responde con código 400 y la lista de errores.
Procesar en WF2	Execute Workflow	Envía los datos válidos al WF2 y espera su resultado.
Responder resultado	Respond to Webhook	Responde 201 si el préstamo se aprobó o 409 si no.

5.1.2 Reglas de validación

Las reglas de validación aplicadas son las siguientes:

Dato	Regla
carne	Formato 0000-00-0000 (expresión regular).
nombre_estudiante	Solo letras y espacios, de 3 a 100 caracteres.
telefono	8 dígitos. Se aceptan espacios o guiones (5555-1234), que se eliminan antes de guardar.
codigo_equipo	Formato AAA-000, por ejemplo LAP-001.
cantidad	Número entero entre 1 y 10.
fecha_devolucion	Fecha real en formato AAAA-MM-DD, entre hoy y 30 días después.

Mostrando la Figura de WF1



5.2 WF2 – Procesamiento de préstamos

Objetivo: decidir si el préstamo se puede entregar y, si se puede, registrarlo sin dejar el inventario inconsistente.

Disparador: nodo When Executed by Another Workflow. Solo se ejecuta cuando lo llama el WF1, por lo que nadie puede usarlo desde afuera para saltarse la validación.

Su nodo central ejecuta una única consulta SQL que descuenta el stock solo si alcanza y, en la misma instrucción, registra el préstamo. Al ser una sola operación, PostgreSQL la trata como una transacción: si dos personas piden el último equipo al mismo tiempo, solo una lo obtiene, y si algo falla no se guarda nada a medias. Los datos del estudiante viajan como parámetros separados del SQL, lo que evita la inyección SQL.

Solo puede ejecutarse desde el WF1, nunca directamente desde fuera. Su nodo central ejecuta una única consulta SQL que descuenta el stock solo si alcanza y, en la misma instrucción, registra el préstamo:

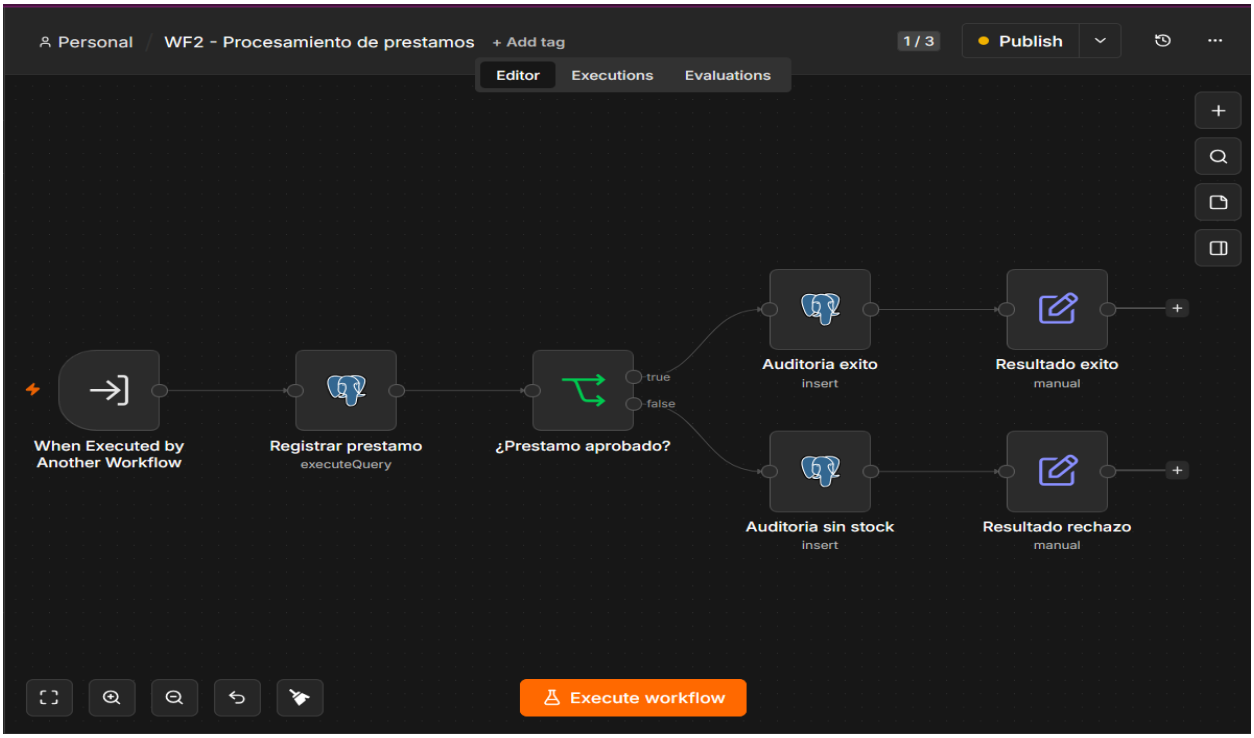
```
WITH equipo AS (
  UPDATE equipos SET stock_disponible = stock_disponible - $2
  WHERE codigo = $1 AND activo = TRUE AND stock_disponible >= $2
  RETURNING id_equipo, nombre, stock_disponible
), nuevo AS (
  INSERT INTO prestamos (carne, nombre_estudiante, telefono, id_equipo, cantidad,
                         fecha_devolucion_esperada)
  SELECT $3, $4, $6, id_equipo, $2, $5::date FROM equipo
  RETURNING id_prestamo
```

```
)
SELECT ... AS aprobado, ... AS id_prestamo, ... AS equipo_existe ...;
```

Como todo ocurre en una sola instrucción, PostgreSQL la ejecuta como una transacción: si dos solicitudes compiten por la última unidad, solo una la obtiene, y si algo falla no se guarda nada a medias.

5.2.1 Nodos

Nodo	Tipo	Función
When Executed by Another Workflow	Execute Workflow Trigger	Recibe los datos validados desde el WF1.
Registrar prestamo	Postgres (Execute Query)	Consulta atómica parametrizada descrita arriba.
¿Prestamo aprobado?	If	Separa préstamos aprobados y rechazados.
Auditoria exito	Postgres (Insert)	Registra el préstamo aprobado con su número y el stock restante.
Resultado exito	Edit Fields	Arma la respuesta: estado APROBADO, número de préstamo, equipo, stock restante y fecha de devolución.
Auditoria sin stock	Postgres (Insert)	Registra el rechazo y su motivo: stock insuficiente o equipo inexistente.
Resultado rechazo	Edit Fields	Arma la respuesta: estado RECHAZADO, motivo y stock disponible.



5.3 WF3 – Salida de reportes

Objetivo: dar seguimiento a los préstamos sin que nadie tenga que revisar el cuaderno: detectar los equipos que no se devolvieron a tiempo y dejar un reporte verificable.

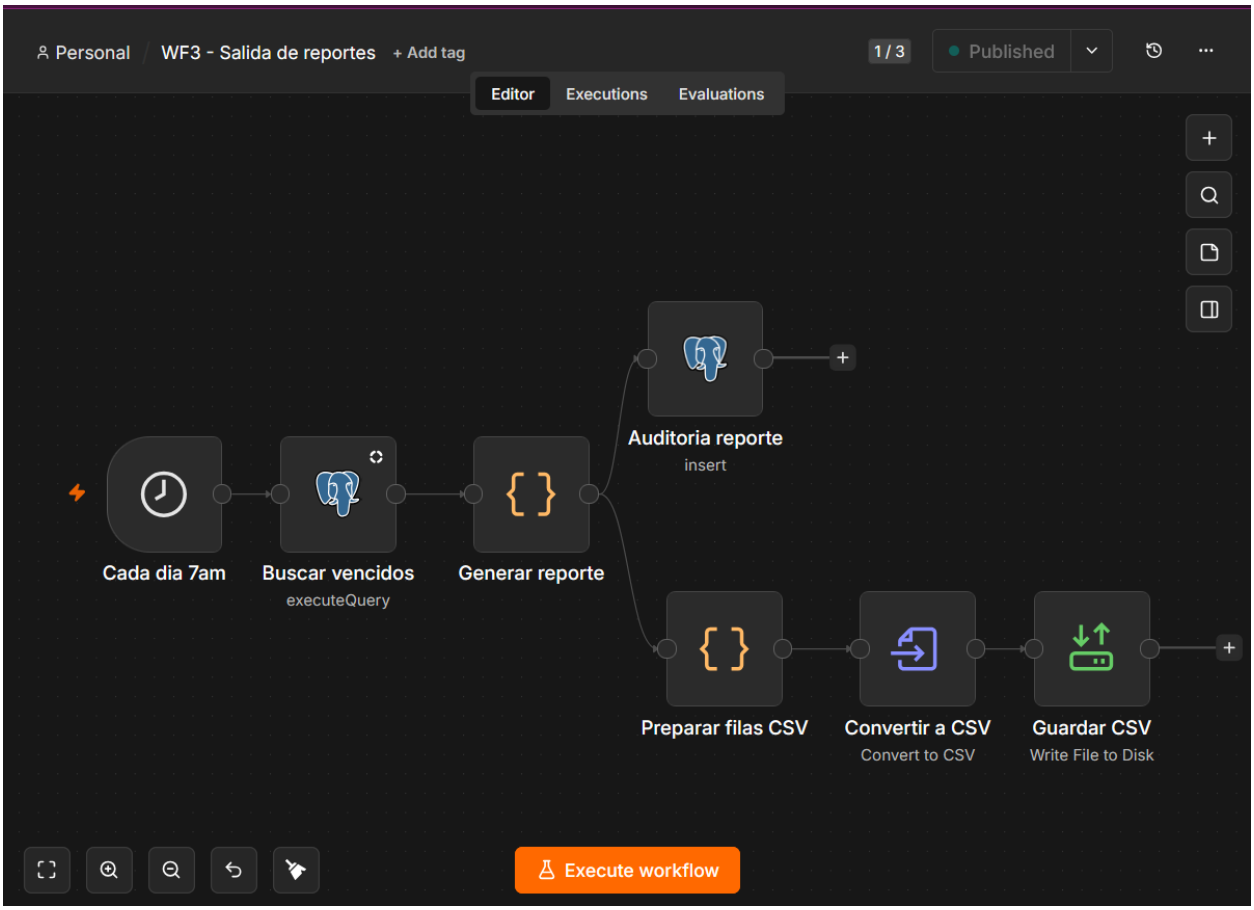
Disparador: nodo Schedule Trigger que lo ejecuta todos los días a las 7:00 a. m. (hora de Guatemala).

El workflow marca como VENCIDO todo préstamo activo cuya fecha de devolución ya pasó y genera dos salidas: un registro con el reporte completo en la tabla auditoria y un archivo CSV en la carpeta reportes, con el nombre reporte_vencidos_AAAA-MM-DD.csv, que se puede abrir en Excel. Si un día no hay vencidos, el reporte se genera igual indicando "Sin préstamos vencidos".

Se ejecuta automáticamente todos los días a las 7:00 a. m. Marca como VENCIDO todo préstamo activo cuya fecha de devolución ya pasó y produce un reporte con dos salidas verificables: un registro en la tabla auditoria y un archivo CSV en la carpeta reportes/, con el nombre reporte_vencidos_AAAA-MM-DD.csv.

5.3.1 Nodos

Nodo	Tipo	Función
Cada dia 7am	Schedule Trigger	Inicia el workflow diariamente a las 7:00 a. m.
Buscar vencidos	Postgres (Execute Query)	Marca los vencidos y devuelve la lista con los días de atraso. Usa "Always Output Data" para continuar aunque no haya vencidos.
Generar reporte	Code	Resume el resultado: fecha, total de vencidos, nuevos del día y detalle.
Auditoria reporte	Postgres (Insert)	Guarda el reporte completo en la bitácora.
Preparar filas CSV	Code	Convierte el reporte en una fila por préstamo vencido.
Convertir a CSV	Convert to File	Genera el archivo CSV con encabezados.
Guardar CSV	Read/Write Files from Disk	Escribe el archivo en /reportes.



5.4 WF4 – Manejo de errores

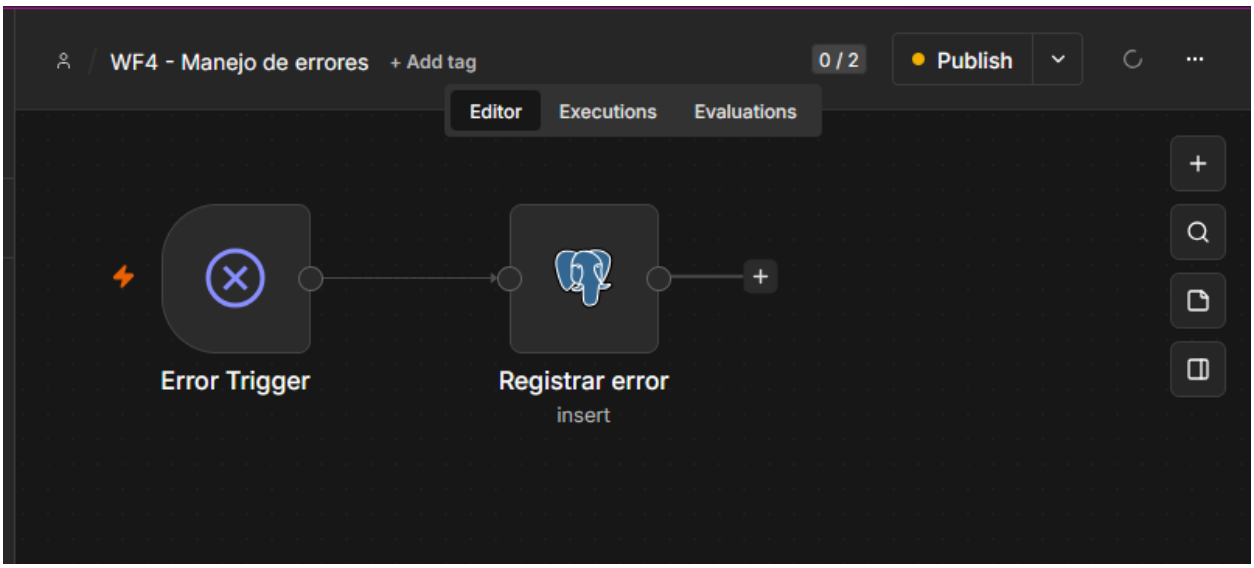
Objetivo: que ningún fallo pase desapercibido.

Disparador: nodo Error Trigger. Está configurado como Error Workflow del WF1, el WF2 y el WF3, así que n8n lo ejecuta automáticamente cuando cualquiera de ellos falla.

Registra en la tabla errores el workflow, el nodo exacto que falló, el mensaje, el número de ejecución y el detalle técnico. Como el WF1 llama al WF2, un fallo en el WF2 queda registrado dos veces: donde se originó y donde se recibió. Así se puede seguir el recorrido completo del error.

Está configurado como Error Workflow del WF1, el WF2 y el WF3. Cuando cualquiera de ellos falla en una ejecución de producción, n8n lo activa automáticamente con el detalle del error.

Nodo	Tipo	Función
Error Trigger	Error Trigger	Se activa cuando falla un workflow vigilado.
Registrar error	Postgres (Insert)	Guarda el workflow, el nodo que falló, el mensaje, el número de ejecución y el detalle técnico en la tabla errores.



5.5 Resumen de nodos utilizados

En total el sistema usa doce tipos de nodo distintos, el doble del mínimo de seis que pide el proyecto: Webhook, Code, If, Postgres, Respond to Webhook, Execute Workflow, Execute Workflow Trigger, Edit Fields, Schedule Trigger, Convert to File, Read/Write Files from Disk y Error Trigger.

6. Cómo trabajan juntos

1. Una solicitud llega al webhook del WF1 con la clave de acceso.
2. El WF1 valida los datos. Si son inválidos, registra el rechazo y responde 400.
3. Si son válidos, el WF1 llama al WF2, que intenta registrar el préstamo y descontar el stock.
4. El WF2 registra el resultado en la auditoría y lo devuelve al WF1, que responde 201 (aprobado) o 409 (sin stock o equipo inexistente).
5. Cada mañana, el WF3 revisa los préstamos vencidos y genera el reporte.
6. Si cualquiera de los tres falla, el WF4 registra el error.

Código	Significado
201	Préstamo aprobado y registrado.
400	Uno o más datos no cumplen las validaciones.
403	Falta la clave X-API-Key o es incorrecta.
409	El equipo no existe, está inactivo o no hay stock suficiente.

7. Manejo de errores y de los datos

7.1 Errores esperados

Los errores previsibles se manejan dentro del propio flujo y se responden con el código HTTP estándar que corresponde, para que quien envía la solicitud sepa exactamente qué pasó:

Código	Situación	Registro
201	Préstamo aprobado y registrado.	auditoria: PRESTAMO_REGISTRADO / EXITO
400	Uno o más datos no cumplen las validaciones.	auditoria: SOLICITUD_PRESTAMO / RECHAZADO
403	Falta la clave X-API-Key o es incorrecta.	Rechazado por n8n antes de ejecutar el flujo
409	El equipo no existe, está inactivo o no hay stock suficiente.	auditoria: PRESTAMO_RECHAZADO / RECHAZADO

7.2 Errores inesperados

Los fallos no previstos (por ejemplo, que la base de datos no responda) los captura el WF4. Como el WF1 llama al WF2, un fallo en el WF2 queda registrado dos veces: en el nodo donde se originó y en el nodo del WF1 que lo recibió. Así se puede seguir el recorrido completo del error.

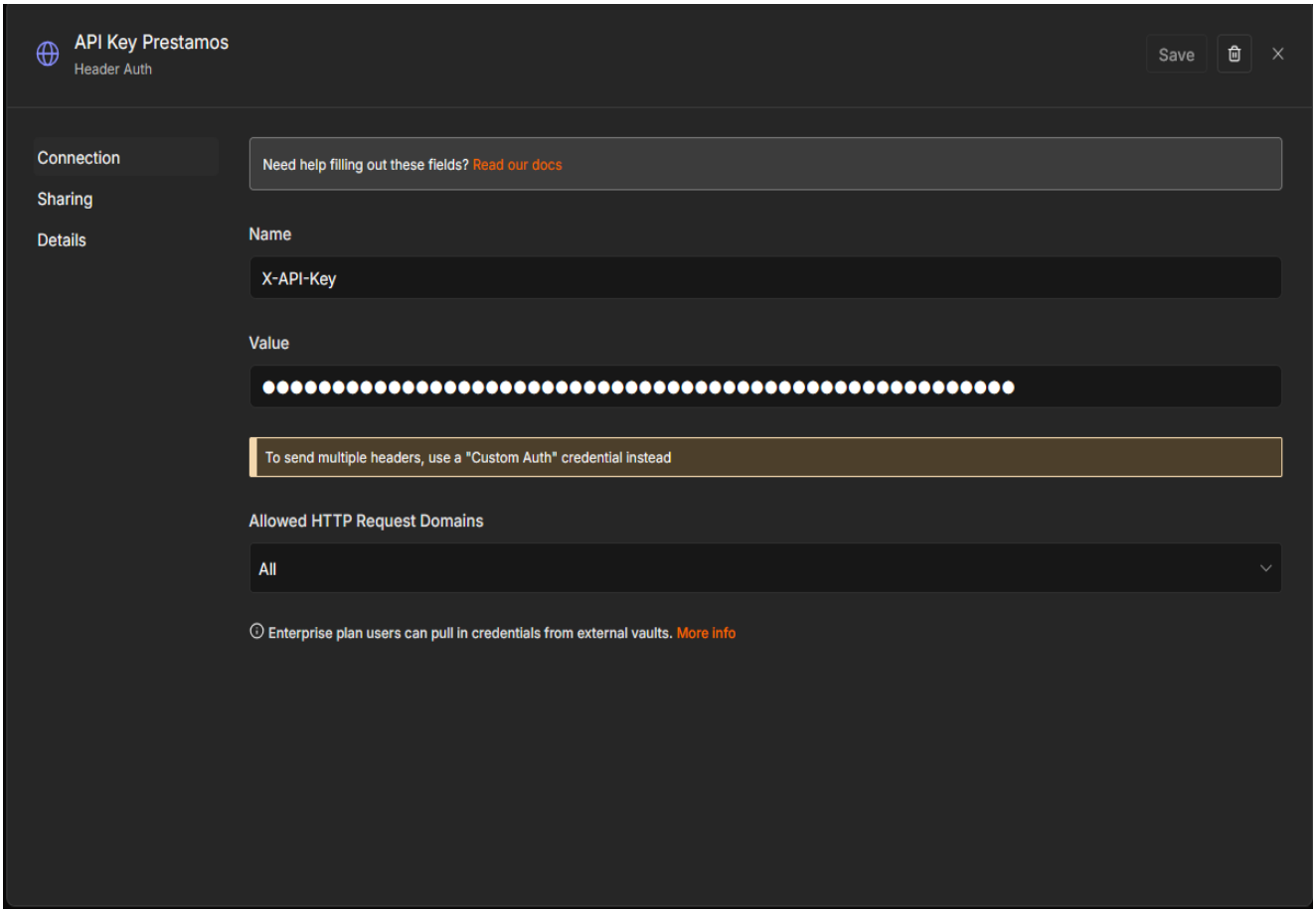
7.3 Consistencia de la información

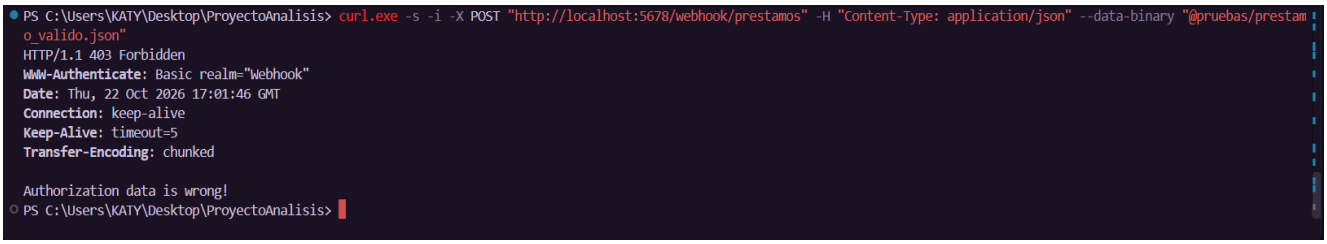
La consistencia se garantiza en tres niveles: la validación en el WF1 impide que entren datos mal formados; la consulta atómica del WF2 impide que el stock y los préstamos queden desalineados; y las restricciones CHECK de PostgreSQL actúan como última barrera aunque un workflow tuviera un defecto.

8. Consideraciones de seguridad

Medida	Implementación	Riesgo que reduce
Credenciales en variables de entorno	Usuario y contraseña de PostgreSQL y clave de cifrado de n8n en .env, leídas por docker-compose.yml. El .env está en .gitignore y el repositorio solo incluye .env.example.	Exposición de contraseñas en el código o en GitHub.
Control de acceso	El webhook exige la cabecera X-API-Key (Header Auth). Sin la clave correcta n8n responde 403.	Solicitudes de terceros no autorizados.
Validación de entradas	Expresiones regulares y rangos para cada campo antes de tocar la base de datos.	Datos corruptos e inyección de contenido malicioso.
Consultas parametrizadas	Los datos viajan separados del SQL ($1…$6) o mediante el modo Insert de n8n.	Inyección SQL.
Credenciales cifradas en n8n	N8N_ENCRYPTION_KEY cifra las credenciales guardadas. Los workflows exportados solo contienen su nombre.	Robo de credenciales desde la configuración de n8n.
Acceso a archivos restringido	N8N_RESTRICT_FILE_ACCESS_TO limita a n8n a la carpeta /reportes.	Lectura o escritura de archivos del sistema.
Sub-workflow no expuesto	El WF2 solo se ejecuta cuando lo llama el WF1.	Registro de préstamos saltándose la validación.
Datos personales mínimos	Solo se guardan el carné, el nombre y el teléfono, lo indispensable para el préstamo y para contactar al estudiante.	Exposición innecesaria de información de estudiantes.
Clave fuera del código (aplicación web)	La aplicación web lee la clave del webhook solo de la variable de entorno N8N_API_KEY. Si falta, se niega a arrancar.	Exposición de la clave en el código o en GitHub.
Validación en dos capas	La aplicación web aplica las mismas reglas del WF1 antes de llamar a n8n.	Datos que intenten saltarse una sola validación.

Medida	Implementación	Riesgo que reduce
Tiempo máximo de espera	La aplicación corta la conexión con n8n a los 10 segundos y muestra un mensaje claro.	Bloqueo de la aplicación si n8n deja de responder.
Formato del teléfono	Restricción CHECK en la base de datos que solo acepta 8 dígitos, además de la validación en la aplicación y en el WF1.	Teléfonos mal registrados aunque fallara una validación.
Contenedor sin privilegios	La aplicación web corre con el usuario appuser, en una imagen que solo contiene Java y el .jar.	Que una falla en la aplicación dé control administrativo del contenedor.
Comprobantes no falsificables	Los datos del comprobante PDF se guardan en la sesión del servidor a partir de lo que confirmó n8n; el navegador nunca los envía.	Comprobantes alterados o de préstamos inexistentes.





9. Pruebas realizadas

Las pruebas se ejecutaron entre el 21 y el 24 de octubre de 2026. Los archivos de entrada están en la carpeta pruebas/ del repositorio.

#	Caso	Entrada	Resultado esperado	Resultado obtenido
1	Datos inválidos	prestamo_invalido.json (6 campos incorrectos)	400 con 6 errores; auditoría RECHAZADO	Correcto
2	Préstamo válido	prestamo_valido.json (1 proyector PRO-001)	201; stock de 2 a 1	Correcto
3	Segundo préstamo válido	prestamo_valido.json	201; stock de 1 a 0	Correcto
4	Sin stock	prestamo_valido.json	409 "Stock insuficiente"	Correcto
5	Consistencia	Consulta a equipos, prestamos y auditoria	2 prestados + 0 disponibles = 2 totales; 4 eventos en auditoría	Correcto
6	Error simulado	Tabla prestamos renombrada temporalmente	WF4 registra el error en WF2 y en WF1; datos intactos	Correcto
7	Préstamo vencido	crear_vencido.sql y ejecución del WF3	Préstamo pasa a VENCIDO (7 días); CSV generado	Correcto
8	Acceso sin clave	Solicitud sin cabecera X-API-Key	403	Correcto
9	Teléfono con guion	prestamo_valido_laptop.json (teléfono 5555-1234)	201; el teléfono se guarda como 55551234	Correcto

```
PS C:\Users\KATY\Desktop\ProyectoAnalisis> curl.exe -i -X POST "http://localhost:5678/webhook/prestamos" -H "Content-Type: application/json" -H "X-API-Key: $env:API_KEY
" --data-binary "@pruebas/prestamo_invalido.json"
HTTP/1.1 400 Bad Request
Content-Security-Policy: sandbox allow-downloads allow-forms allow-modals allow-orientation-lock allow-pointer-lock allow-popups allow-popups-to-escape-sandbox allow-pr
esentation allow-scripts allow-top-navigation-by-user-activation allow-top-navigation-to-custom-protocols
Content-Type: application/json; charset=utf-8
Content-Length: 382
ETag: W/"17e-ficevejgiXyvvWbj0Fx65FiZsyA"
Vary: Accept-Encoding
Date: Thu, 22 Oct 2026 20:03:26 GMT
Connection: keep-alive
Keep-Alive: timeout=5

{"estado":"RECHAZADO","mensaje":"La solicitud tiene datos invalidos","errores":["Carné inválido. Formato esperado: 0000-00-0000","Nombre inválido. Solo letras y espacio
s (3 a 100 caracteres)","Código de equipo inválido. Formato esperado: AAA-000","Cantidad inválida. Debe ser un número entero entre 1 y 10","La fecha de devolución debe
estar entre 2026-10-22 y 2026-11-21"]}
PS C:\Users\KATY\Desktop\ProyectoAnalisis>
```


```
PS C:\Users\KATY\Desktop\ProyectoAnalisis> curl.exe -i -X POST "http://localhost:5678/webhook/prestamos" -H "Content-Type: application/json" -H "X-API-Key: $env:API_KEY
" --data-binary "@pruebas/prestamo_invalido.json"
HTTP/1.1 400 Bad Request
Content-Security-Policy: sandbox allow-downloads allow-forms allow-modals allow-orientation-lock allow-pointer-lock allow-popups allow-popups-to-escape-sandbox allow-pr
esentation allow-scripts allow-top-navigation-by-user-activation allow-top-navigation-to-custom-protocols
Content-Type: application/json; charset=utf-8
Content-Length: 382
ETag: W/"17e-ficevejgiXyvvWbj0Fx65FiZsyA"
Vary: Accept-Encoding
Date: Thu, 22 Oct 2026 20:03:26 GMT
Connection: keep-alive
Keep-Alive: timeout=5

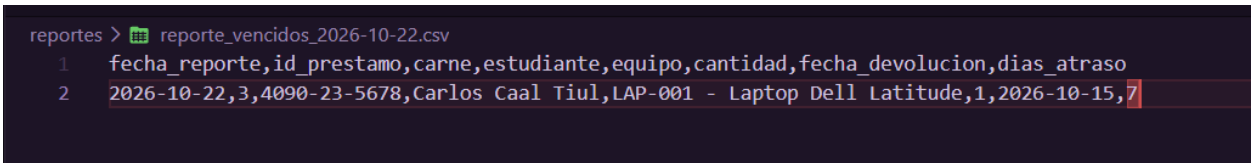
{"estado":"RECHAZADO","mensaje":"La solicitud tiene datos invalidos","errores":["Carné inválido. Formato esperado: 0000-00-0000","Nombre inválido. Solo letras y espacio
s (3 a 100 caracteres)","Código de equipo inválido. Formato esperado: AAA-000","Cantidad inválida. Debe ser un número entero entre 1 y 10","La fecha de devolución debe
estar entre 2026-10-22 y 2026-11-21"]}
PS C:\Users\KATY\Desktop\ProyectoAnalisis>
```


```
PS C:\Users\KATY\Desktop\ProyectoAnalisis> docker exec -it inventario_db psql -U irma_admin -d inventario_lab -c "SELECT id_auditoria, fecha, workflow, accion, resultad
o FROM auditoria ORDER BY id_auditoria;"
 id_auditoria |           fecha            |             workflow             |       accion        | resultado
--------------+----------------------------+----------------------------------+---------------------+-----------
            1 | 2026-10-21 17:24:40.537014 | WF1 - Ingesta de prestamos       | SOLICITUD_PRESTAMO  | RECHAZADO
            2 | 2026-10-21 22:41:31.937784 | WF2 - Procesamiento de prestamos | PRESTAMO_REGISTRADO | EXITO
            3 | 2026-10-21 22:43:56.446106 | WF2 - Procesamiento de prestamos | PRESTAMO_REGISTRADO | EXITO
            4 | 2026-10-21 22:45:40.536453 | WF2 - Procesamiento de prestamos | PRESTAMO_RECHAZADO  | RECHAZADO
            5 | 2026-10-22 08:29:04.570097 | WF3 - Salida de reportes         | REPORTE_VENCIDOS    | EXITO
            6 | 2026-10-22 08:53:54.301595 | WF3 - Salida de reportes         | REPORTE_VENCIDOS    | EXITO
            7 | 2026-10-22 14:02:21.805519 | WF2 - Procesamiento de prestamos | PRESTAMO_REGISTRADO | EXITO
            8 | 2026-10-22 14:03:26.065634 | WF1 - Ingesta de prestamos       | SOLICITUD_PRESTAMO  | RECHAZADO
(8 rows)


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug inventario_db
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\KATY\Desktop\ProyectoAnalisis>
```

```
PS C:\Users\KATY\Desktop\ProyectoAnalisis> docker exec -it inventario_db psql -U irma_admin -d inventario_lab -c "SELECT id_error, fecha, workflow, nodo, mensaje FROM e
rrores;"
 id_error |           fecha            |             workflow             |       nodo        |                mensaje
----------+----------------------------+----------------------------------+-------------------+---------------------------------------
        1 | 2026-10-21 23:17:30.676317 | WF2 - Procesamiento de prestamos | Registrar prestamo | relation "prestamos" does not exist
        2 | 2026-10-21 23:17:30.687704 | WF1 - Ingesta de prestamos       | Procesar en WF2    | relation "prestamos" does not exist
(2 rows)


What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug inventario_db
    Learn more at https://docs.docker.com/go/debug-cli/
PS C:\Users\KATY\Desktop\ProyectoAnalisis>
```



10. Instalación y reproducibilidad

El archivo README.md del repositorio contiene las instrucciones completas. En resumen:

1. Instalar Docker Desktop y Git.
2. Clonar el repositorio y copiar .env.example como .env, completando sus valores.
3. Ejecutar docker compose up -d. PostgreSQL crea las tablas automáticamente.
4. Entrar a http://localhost:5678, crear la cuenta de n8n y las credenciales Postgres Inventario y API Key Prestamos.
5. Importar los workflows de la carpeta workflows/ en el orden WF04, WF02, WF01 y WF03, asignar credenciales, configurar el WF4 como Error Workflow y publicar.
6. Ejecutar los casos de la carpeta pruebas/.
7. La aplicación web se levanta automáticamente como tercer contenedor con docker compose up -d; solo requiere la clave N8N_API_KEY en el archivo .env.
8. El historial de commits del repositorio documenta la evolución del proyecto, desde el entorno inicial (v0.2) hasta la versión con el teléfono del estudiante (v1.4).

11. Aplicación web de préstamos

Para que el personal del laboratorio registre préstamos sin usar la terminal, se desarrolló una aplicación web en el módulo app-prestamos del repositorio. La aplicación no reemplaza a los workflows: es una nueva puerta de entrada que envía cada solicitud al webhook del WF1, de modo que la lógica, la auditoría y el manejo de errores de n8n se siguen aplicando.

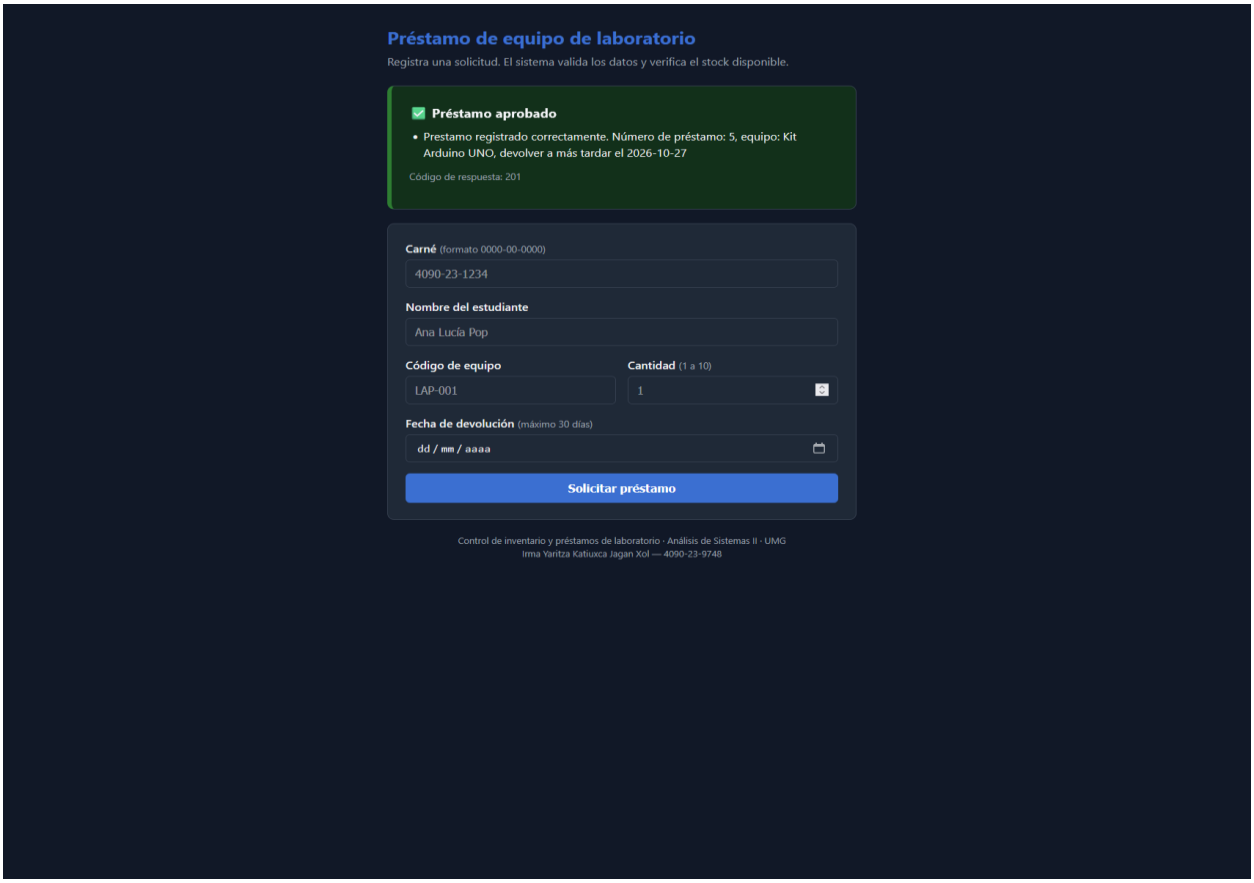
11.1 Tecnologías y arquitectura

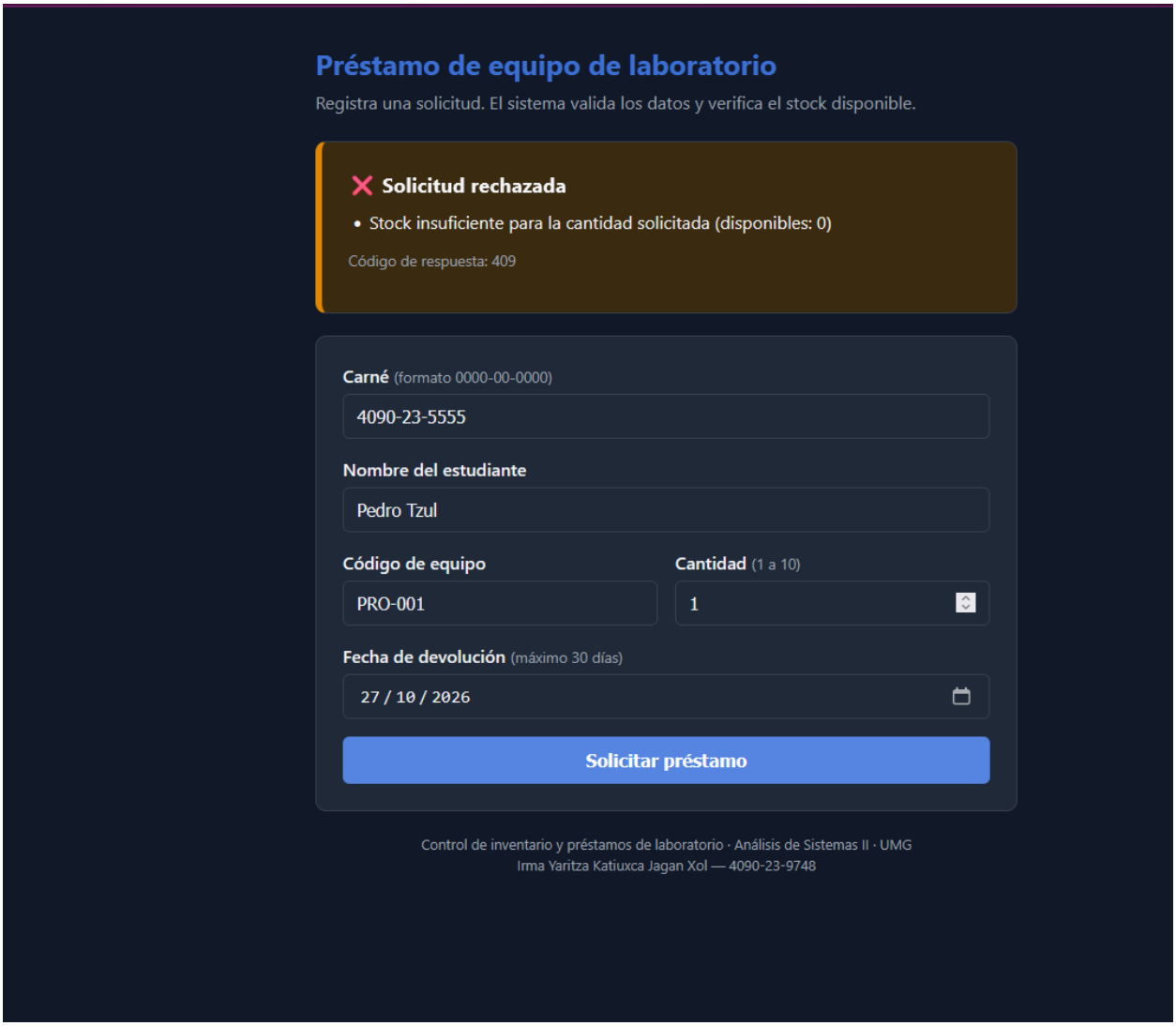
Se construyó con Spring Boot 4.1, que incluye su propio servidor web (Tomcat) en el puerto 8080, y con Thymeleaf como motor de plantillas HTML. Sigue el patrón Modelo–Vista–Controlador (MVC): el controlador recibe el formulario, un servicio aplica las reglas de negocio y la vista muestra el resultado. Las clases reciben sus dependencias por inyección de dependencias, lo que permite reemplazar cualquier pieza por una simulación durante las pruebas.

Clase	Responsabilidad
PrestamoController	Atiende GET y POST de /prestamos, entrega el comprobante en /prestamos/comprobante y elige la vista que se muestra.
FormularioPrestamo	Recibe los campos del formulario como texto, para que cualquier dato mal escrito llegue al validador y reciba un mensaje claro.
ServicioPrestamos	Valida la solicitud y solo si es válida la envía a n8n. Convierte los fallos de comunicación en un error controlado (503).
ValidadorPrestamo	Aplica las mismas seis reglas del nodo Validar datos del WF1.
PrestamoGateway y N8nPrestamoGateway	Interfaz y cliente HTTP (RestClient) que llama al webhook con la cabecera X-API-Key e interpreta las respuestas 201, 400, 403 y 409.
ConfiguracionPrestamos	Crea y conecta las piezas, y lee la URL del webhook y la clave desde la configuración.
GeneradorComprobantePdf	Dibuja el comprobante del préstamo en PDF con Apache PDFBox.
ComprobantePrestamo y DetallePrestamo	Records con los datos del comprobante y con el detalle del préstamo aprobado que devuelve n8n.
prestamo.html	Plantilla Thymeleaf con el formulario y el resultado en verde, naranja o rojo.

11.2 Flujo de una solicitud

- El usuario llena el formulario en http://localhost:8080/prestamos, incluido el número de celular del estudiante, y lo envía.
- El controlador convierte el formulario en una solicitud y la entrega al servicio.
- El servicio la valida. Si es inválida, responde de inmediato con la lista de errores, sin contactar a n8n.
- Si es válida, el gateway la envía al webhook del WF1, que la procesa junto con el WF2.
- La respuesta se muestra en la página: verde si se aprobó, naranja si se rechazó y rojo si n8n no respondió. Si se rechaza, los datos se conservan para que el usuario solo corrija lo necesario.
- Si el préstamo se aprobó, aparece el botón Descargar comprobante (PDF).


Préstamo de equipo de laboratorio
Registra una solicitud. El sistema valida los datos y verifica el stock disponible.
Préstamo aprobado
Prestamo registrado correctamente. Número de préstamo: 5, equipo: Kit Arduino UNO, devolver a más tardar el 2026-10-27
Código de respuesta: 201
Carné (formato 0000-00-0000)
4090-23-1234
Nombre del estudiante
Ana Lucía Pop
Código de equipo
LAP-001
Cantidad (1 a 10)
1
Fecha de devolución (máximo 30 días)
dd / mm / aaaa
Solicitar préstamo
Control de inventario y préstamos de laboratorio · Análisis de Sistemas II - UMG
Irma Yaritza Katiuxca Jagan Xol — 4090-23-9748



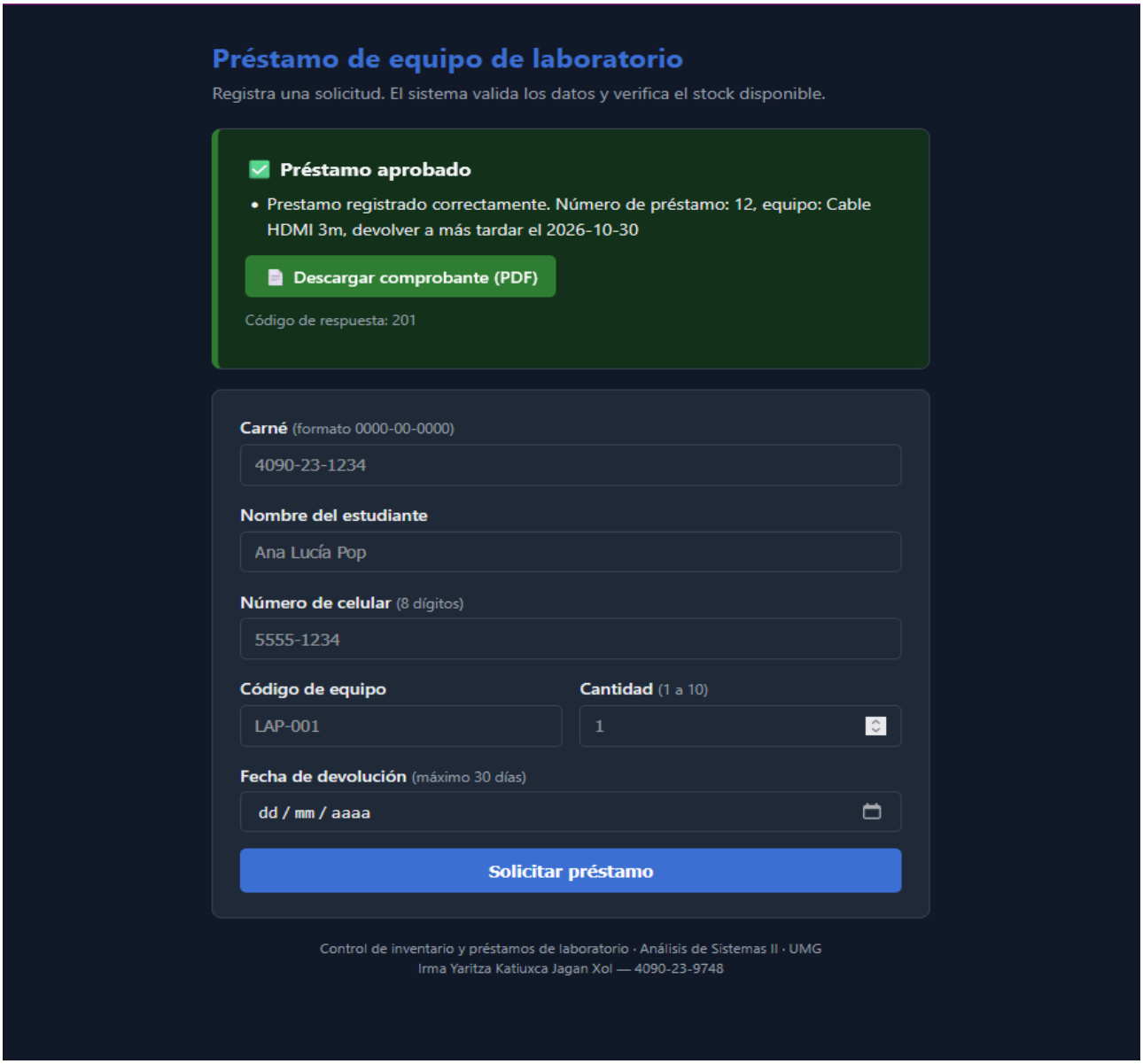
11.3 Seguridad de la aplicación

- La clave del webhook se lee únicamente de la variable de entorno N8N_API_KEY. Si no está configurada, la aplicación se niega a arrancar e indica qué falta, en lugar de funcionar a medias.
- La clave vive solo en el servidor: el navegador nunca la recibe.
- La validación se aplica en dos capas, en la aplicación y en el WF1, de modo que nadie puede saltarse las reglas enviando datos directamente al webhook.
- La conexión con n8n tiene un tiempo máximo de espera de 10 segundos. Si se supera, el usuario recibe un mensaje claro en lugar de una página bloqueada.
- El comprobante no se puede falsificar: sus datos se guardan en la sesión del servidor a partir de lo que confirmó n8n, y el navegador nunca los envía.
- En Docker, la aplicación corre con un usuario sin privilegios (appuser).

11.4 Comprobante del préstamo en PDF

Cuando un préstamo se aprueba, la página muestra el botón Descargar comprobante (PDF). El comprobante, de tamaño carta, contiene el número de préstamo, la fecha y hora de registro, el carné, el nombre y el teléfono del estudiante, el código y el nombre del equipo, la cantidad y la fecha de devolución, además del compromiso de devolución y las líneas para las firmas del encargado y del estudiante. Se imprime, se firma y queda archivado en el laboratorio.

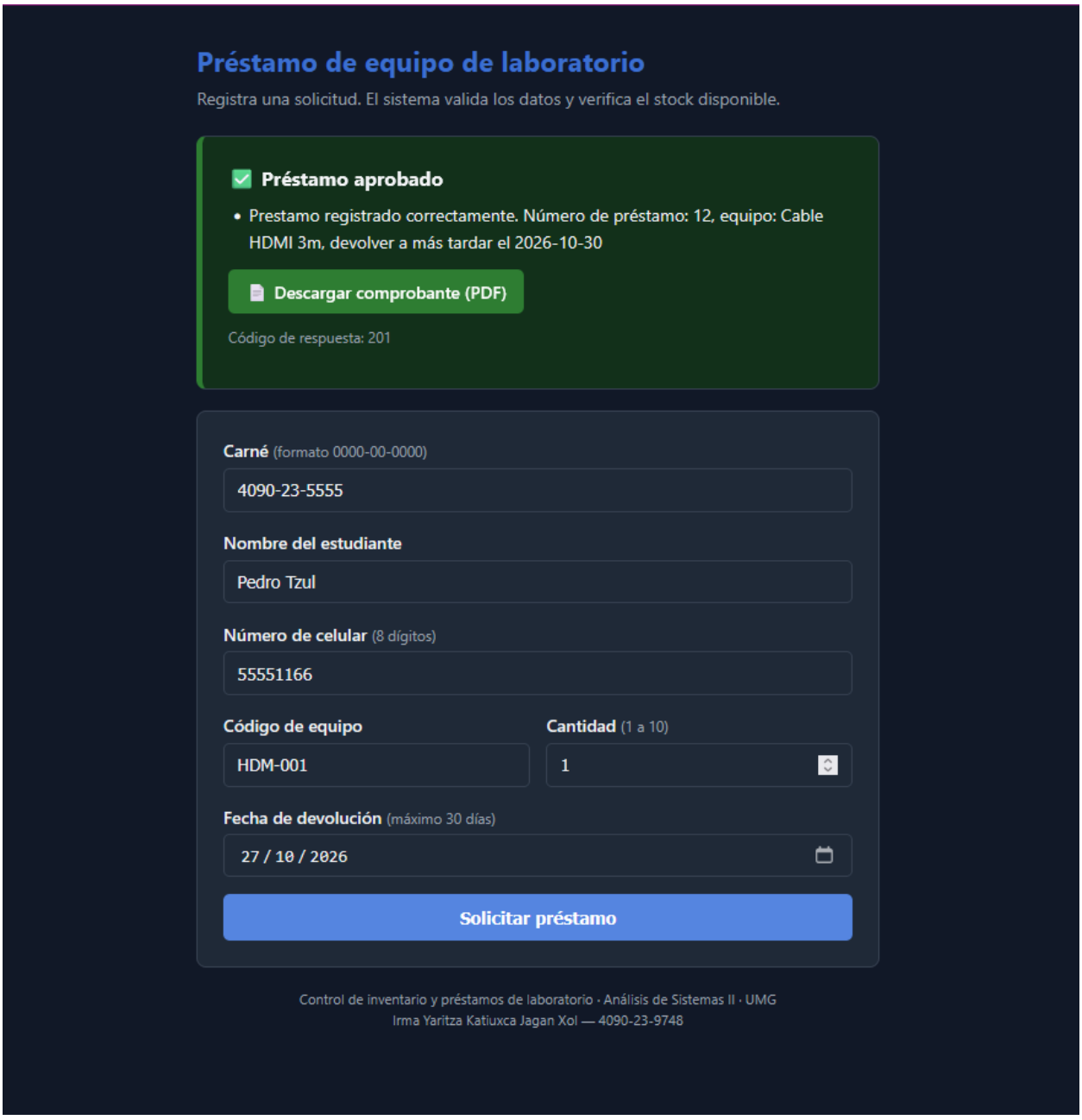
Se genera con Apache PDFBox, usando fuentes estándar de PDF que admiten tildes y ñ. Para evitar comprobantes falsificados, sus datos se guardan en la sesión del servidor a partir de la respuesta de n8n: el navegador solo pide el comprobante de su sesión y nunca envía los datos. Cada solicitud nueva borra el comprobante anterior.



11.5 Número de celular del estudiante

El formulario incluye el número de celular del estudiante. Se valida como número de Guatemala de 8 dígitos y se aceptan espacios o guiones (por ejemplo, 5555-1234), que se eliminan antes de guardarlo. La misma regla se aplica en la aplicación web, en el WF1 y, como última barrera, en una restricción CHECK de la base de datos.

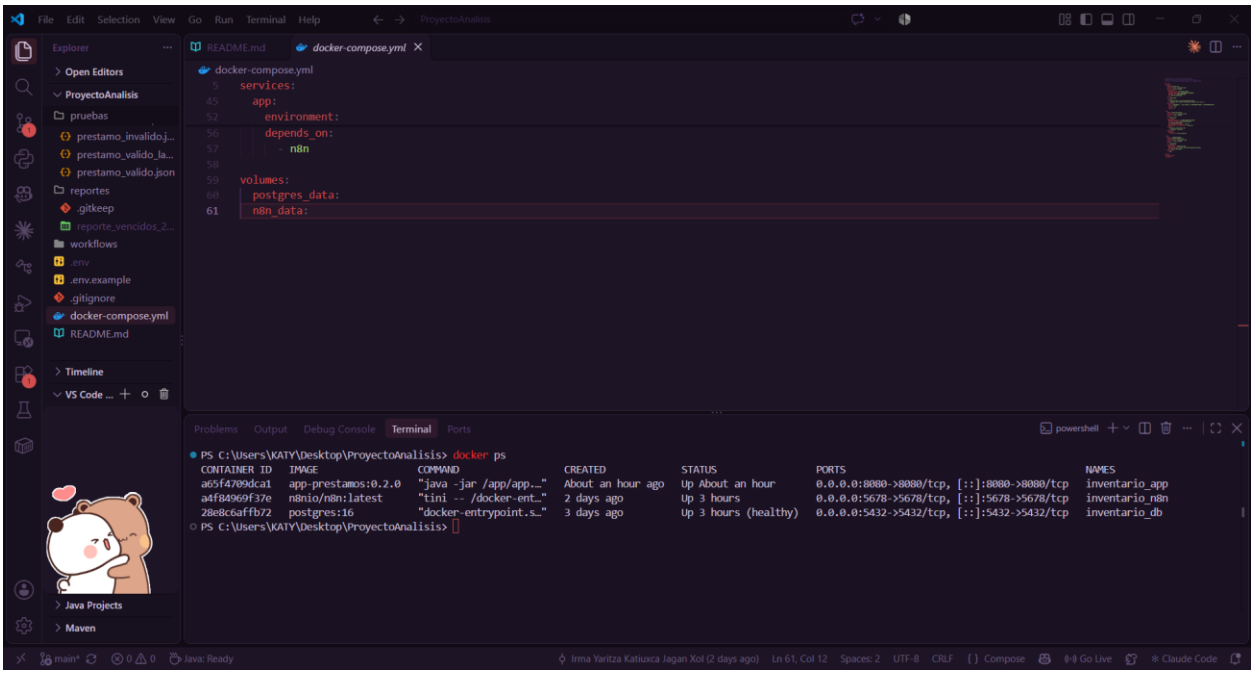
El número se guarda en cada préstamo y no es único, de modo que un mismo estudiante puede registrarlo todas las veces que pida equipo prestado. En el comprobante se imprime como 5555-1234 para que sea fácil de leer.



11.6 Ejecución en Docker

La aplicación web corre en un tercer contenedor, inventario_app, junto a n8n y PostgreSQL. Su imagen se construye con un Dockerfile de dos etapas: la primera compila el proyecto con Maven y la segunda contiene solo Java 21 y el archivo .jar, por lo que la imagen final es pequeña (unos 138 MB) y no incluye el código fuente ni las herramientas de compilación. La aplicación se ejecuta con un usuario sin privilegios (appuser).

El contenedor lee la clave N8N_API_KEY del archivo .env y llama a n8n por su nombre de servicio (http://n8n:5678), dentro de la red interna de Docker. Como los tres contenedores tienen la política restart: unless-stopped, el sistema completo arranca solo cada vez que se inicia Docker Desktop, sin ejecutar ningún comando.





12. Pruebas unitarias automatizadas

Además de las pruebas del sistema de la sección 9, la aplicación web cuenta con 104 pruebas unitarias automatizadas. Se ejecutan con un solo comando (mvn verify), tardan pocos segundos y no necesitan Docker ni n8n, porque las dependencias externas se reemplazan por simulaciones.

12.1 Herramientas

Herramienta	Uso en el proyecto
JUnit 5	Motor de pruebas. Se usaron pruebas parametrizadas para probar cada regla con varios valores válidos, inválidos y límite.
Mockito	Simula el gateway de n8n para probar el servicio, y el servicio para probar el controlador (@MockitoBean).
MockRestServiceServer	Simula el servidor HTTP de n8n y verifica que la petición lleve la URL, el método, la clave y el JSON correctos.
MockMvc	Simula el navegador: abre la página, envía el formulario y revisa el HTML generado.
Apache PDFBox	Además de generar el comprobante, se usa en las pruebas para abrir el PDF generado y leer su texto, comprobando que contenga todos los datos.
JaCoCo	Mide la cobertura del código y hace fallar la construcción si la cobertura de líneas baja del 80 %.

Para que las pruebas que dependen de la fecha den siempre el mismo resultado, el validador recibe un reloj (Clock) que en las pruebas se fija en el 22 de octubre de 2026.

12.2 Casos probados

Clase de prueba	Pruebas	Qué verifica
ValidadorPrestamoTest	59	Las seis reglas con valores válidos, inválidos y límite: por ejemplo, cantidades 1 y 10 aceptadas, 0 y 11 rechazadas, y la fecha 2026-02-30 rechazada por no existir.
ServicioPrestamosTest	10	Que los datos inválidos nunca lleguen a n8n, que los válidos se envíen normalizados una sola vez y que una caída de n8n se convierta en un error 503 controlado.
N8nPrestamoGatewayTest	14	La petición HTTP exacta y la interpretación de cada respuesta de n8n: 201, 400, 403, 409, 500, respuestas vacías y servidor apagado.
PrestamoControllerTest	13	Redirección, formulario vacío, colores del resultado, conservación de los datos al rechazar y cantidades no numéricas, el comprobante guardado en la sesión y la descarga del PDF.
GeneradorComprobantePdfTest	7	Que el PDF sea válido y de una página, que contenga todos los datos con tildes y ñ, y el formato de las fechas y del teléfono.
AppPrestamosApplicationTest	1	Que la aplicación completa arranca y conecta el controlador, el servicio y el gateway real.
Total	**104**	**Todas aprobadas.**

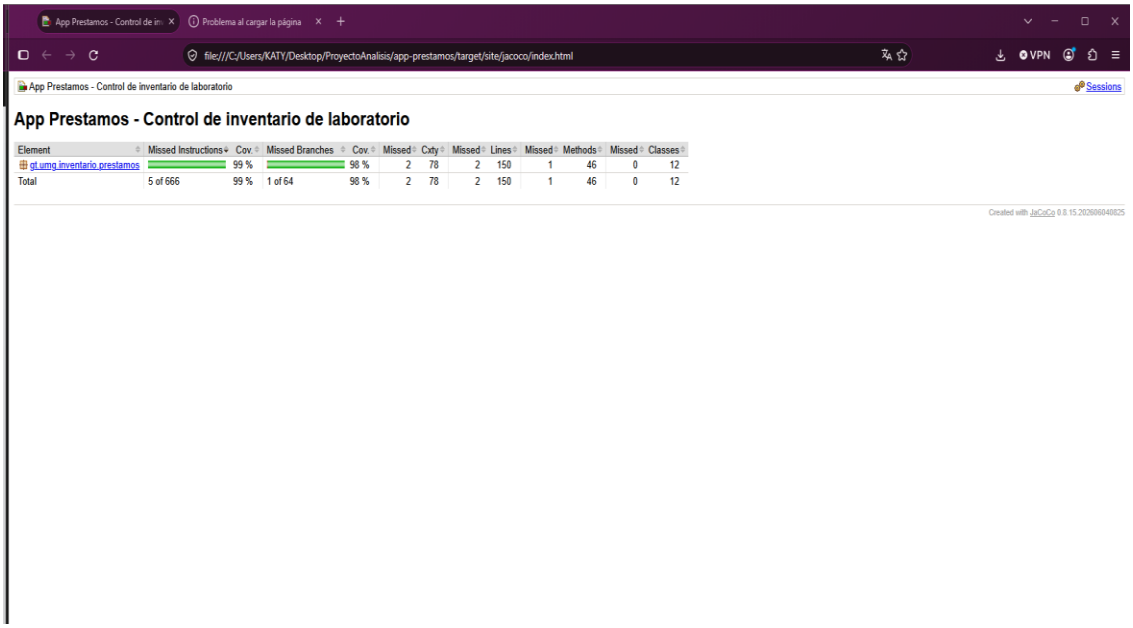
12.3 Cobertura de código

Métrica (JaCoCo)	Resultado
Instrucciones	1 323 de 1 335 (99 %)
Ramas de decisión	83 de 84 (98 %)
Líneas	263 de 267
Métodos	63 de 64
Clases	15 de 15

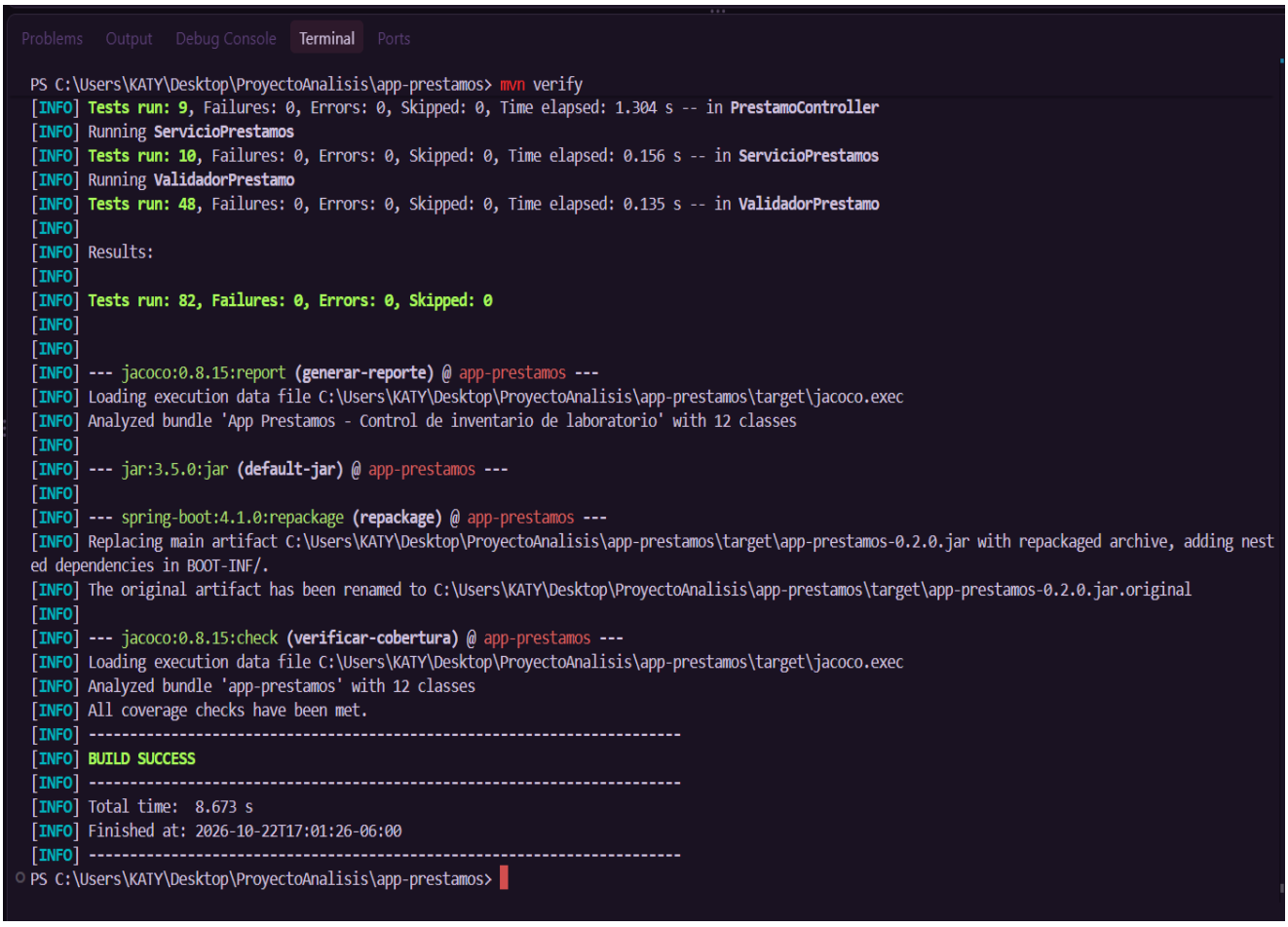
Lo único sin cubrir es el método main que arranca la aplicación, que en las pruebas se inicia por otro camino, y una rama de un caso límite. Toda la lógica de negocio, el gateway y el controlador están cubiertos, muy por encima del mínimo de 80 % configurado.

Las pruebas también ayudaron a detectar y corregir un defecto real: al solicitar un equipo inexistente, la página mostraba «(disponibles: 0)», un dato que confundía al usuario. Se corrigió el mensaje y se agregó una prueba que reproduce ese caso para que no vuelva a aparecer.

Tenemos mostrando el reporte de cobertura de Jacoco



Mostrando la Ejecución de pruebas Unitarias



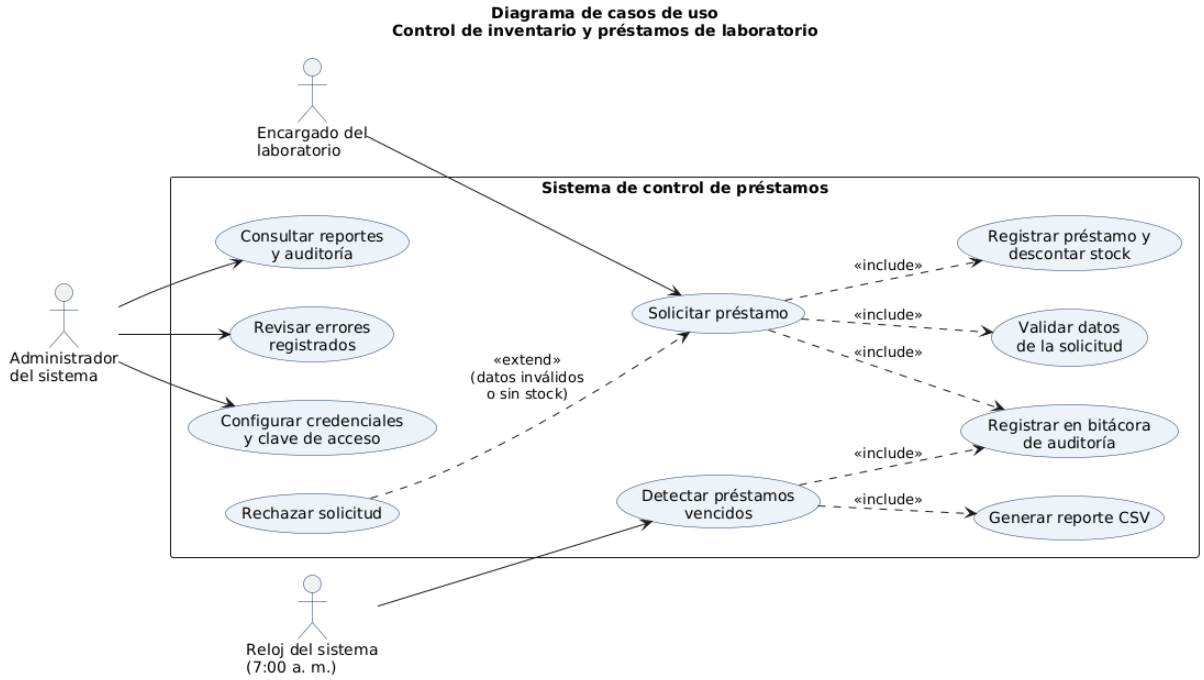
13. Modelado UML

Para documentar el sistema desde distintos puntos de vista se elaboraron cinco diagramas UML con PlantUML. Cada diagrama se escribió como texto, por lo que su código fuente (archivos .puml) está versionado en la carpeta docs/uml del repositorio junto a su imagen, y se puede modificar y volver a generar en cualquier momento.

Diagrama	Tipo de vista	Pregunta que responde
Casos de uso	Funcional	¿Quién usa el sistema y qué puede hacer?
Actividad	Comportamiento	¿Qué pasos sigue un préstamo y qué decisiones se toman?
Secuencia	Interacción	¿En qué orden se comunican las piezas del sistema?
Clases	Estructural	¿Cómo está construido el código de la aplicación web?
Despliegue	Física	¿Dónde se ejecuta cada pieza y cómo se conectan?

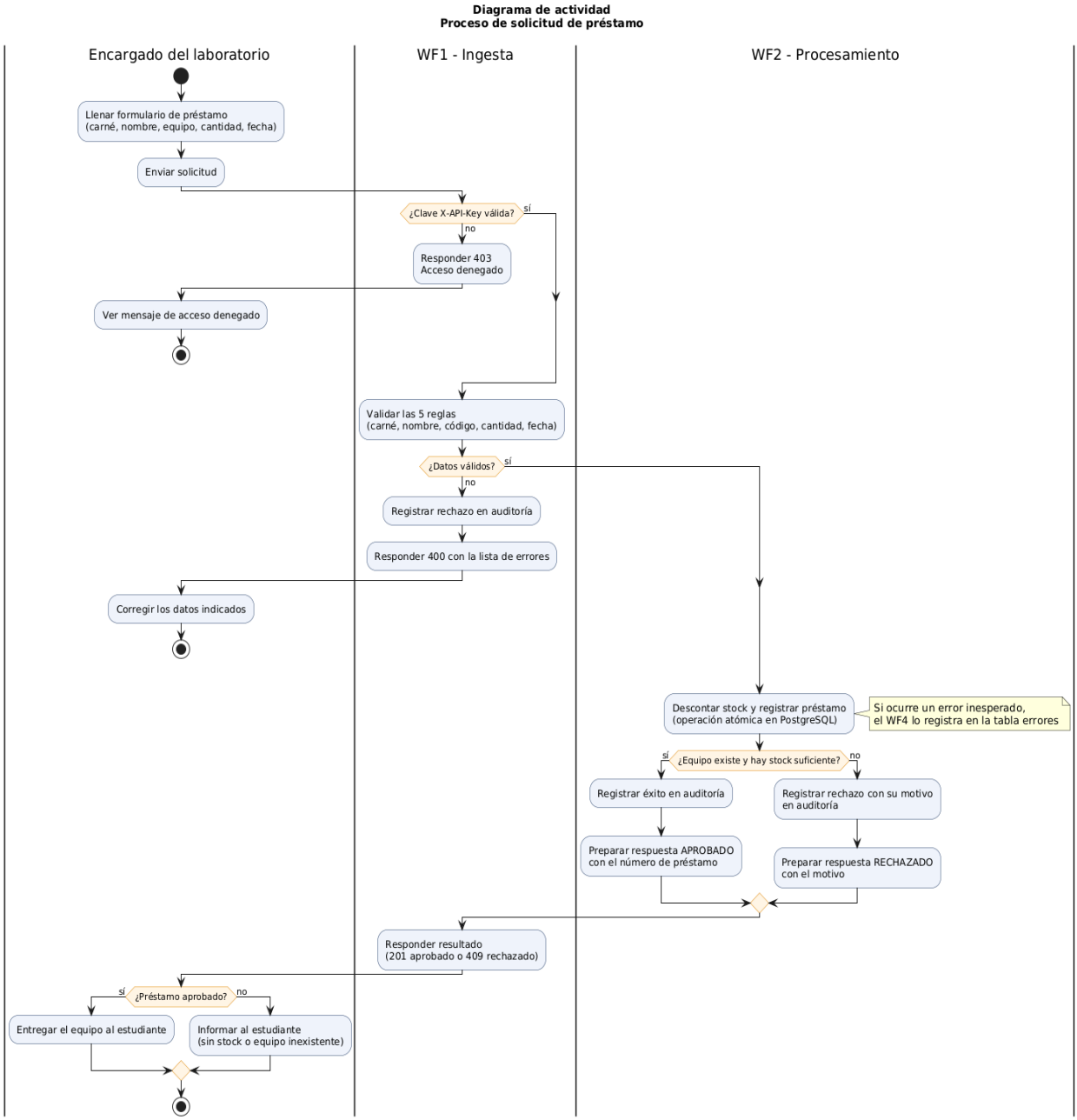
13.1 Diagrama de casos de uso

Identifica a los tres actores del sistema y lo que cada uno puede hacer. El Encargado del laboratorio solicita préstamos; esa acción siempre incluye (<<include>>) validar los datos, registrar el préstamo y dejarlo en la bitácora de auditoría, y solo a veces se extiende (<<extend>>) a rechazar la solicitud, cuando los datos son inválidos o no hay stock. El Reloj del sistema representa al disparador programado del WF3, que cada mañana detecta los préstamos vencidos y genera el reporte CSV. El Administrador consulta reportes, revisa los errores registrados por el WF4 y configura las credenciales.



13.2 Diagrama de actividad

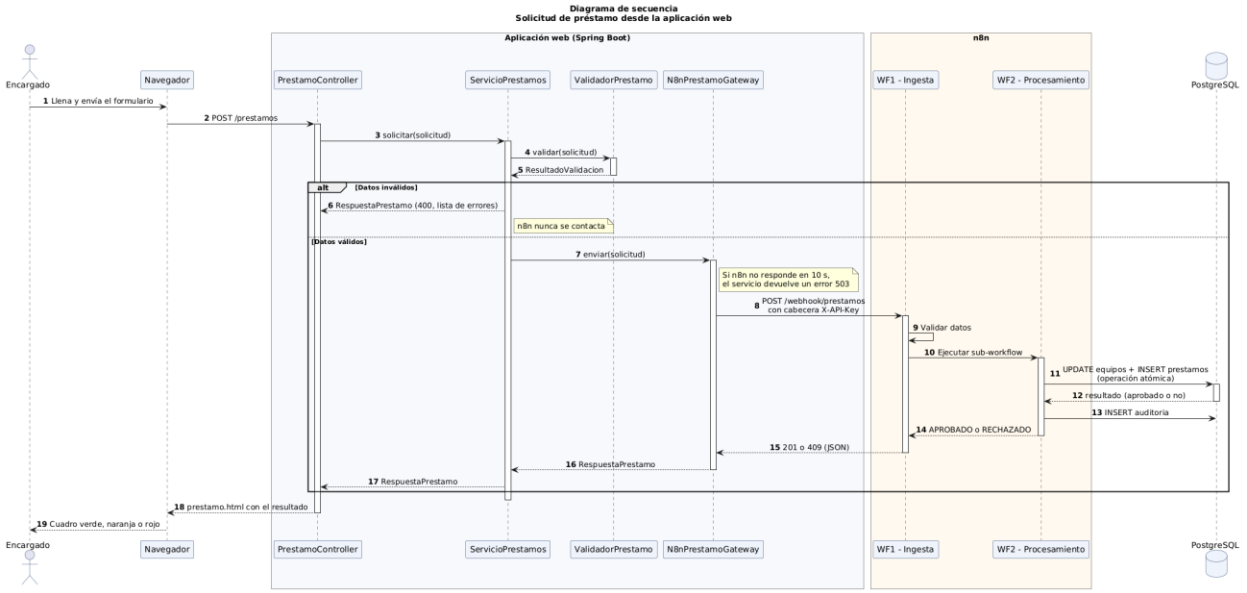
Describe el proceso de una solicitud de préstamo dividido en carriles, uno por responsable: el encargado, el WF1 y el WF2. Los rombos representan las decisiones del flujo: si la clave de acceso es válida, si los datos cumplen las reglas y si hay stock suficiente. Las salidas posibles coinciden con los códigos de respuesta del sistema: 403 sin clave, 400 con datos inválidos, y 201 o 409 según exista stock. La nota indica que el WF4 vigila todo el proceso por si ocurre un error inesperado.



13.3 Diagrama de secuencia

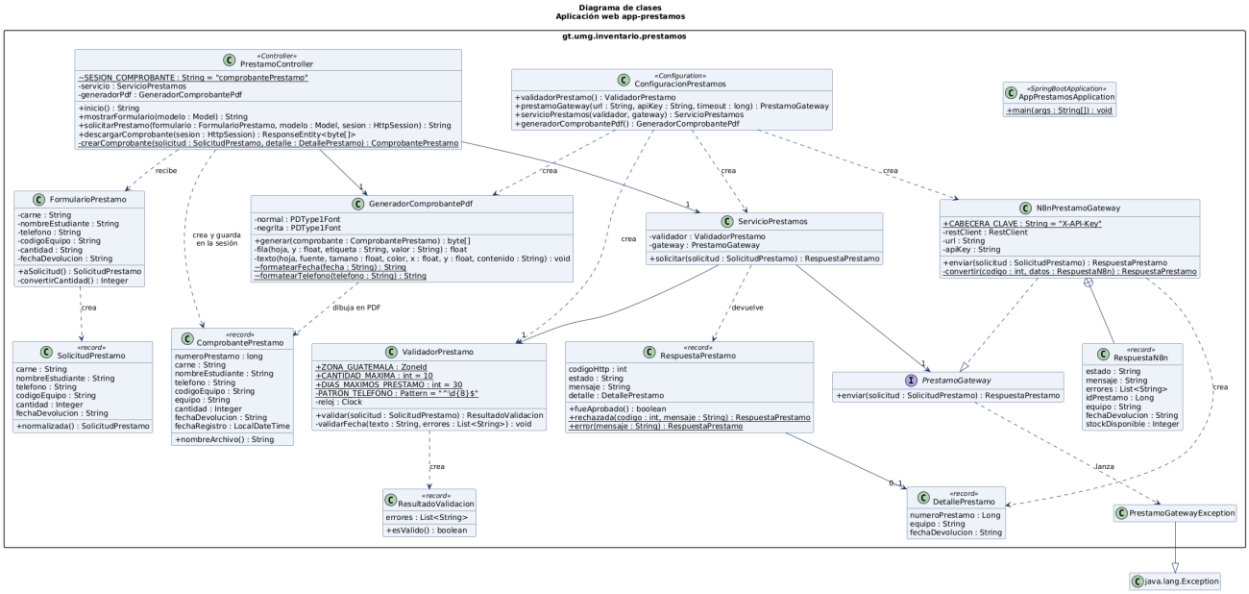
Muestra en orden cronológico los 19 mensajes que intercambian las piezas cuando se solicita un préstamo desde la aplicación web: navegador, controlador, servicio, validador, gateway, WF1, WF2 y PostgreSQL. El fragmento alt presenta las dos alternativas:

Con datos inválidos la secuencia termina dentro de la aplicación web y n8n nunca se contacta (validación en dos capas); con datos válidos la solicitud recorre n8n hasta la base de datos, donde el stock se descuenta y el préstamo se registra en una sola operación atómica.



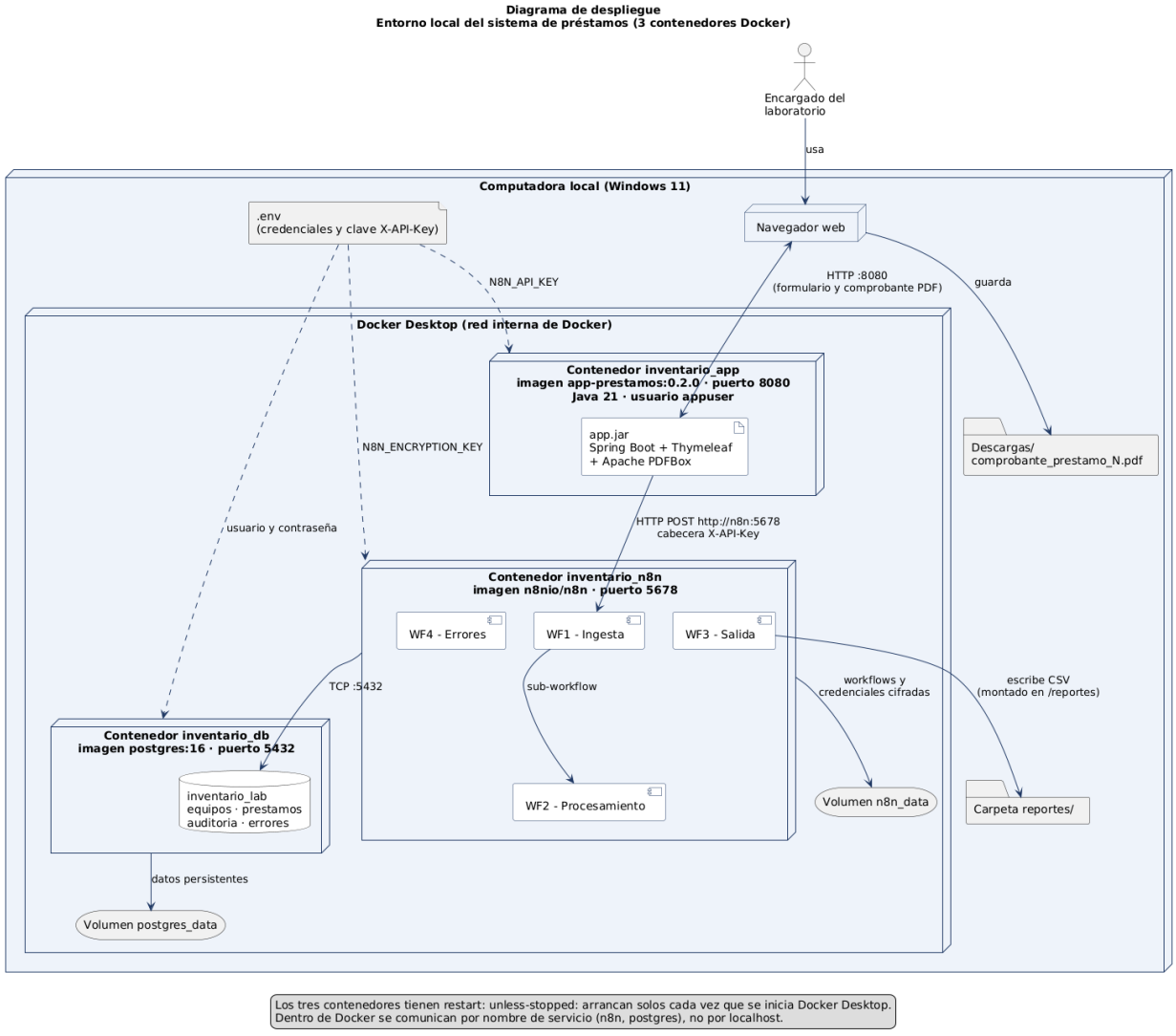
13.4 Diagrama de clases

Presenta las clases de la aplicación web con sus atributos, métodos y relaciones. Las clases de datos se implementaron como records de Java (SolicitudPrestamo, ResultadoValidacion, RespuestaPrestamo y RespuestaN8n), y las anotaciones de Spring aparecen como estereotipos (<<Controller>>, <<Configuration>>). La relación más importante es la de ServicioPrestamos con la interfaz PrestamoGateway: el servicio depende de la interfaz y no de la clase concreta N8nPrestamoGateway, lo que permite que en las pruebas unitarias Mockito la reemplace por una simulación de n8n. Para el comprobante se agregaron GeneradorComprobantePdf, que dibuja el PDF, y los records ComprobantePrestamo y DetallePrestamo; el controlador guarda el comprobante en la sesión y lo entrega en /prestamos/comprobante. El teléfono aparece como atributo del formulario, de la solicitud y del comprobante.



13.5 Diagrama de despliegue

Muestra dónde se ejecuta cada pieza. En la computadora local corren el navegador y Docker Desktop, que contiene tres contenedores: la aplicación web (inventario_app, puerto 8080, con Java 21 y el usuario sin privilegios appuser), n8n (inventario_n8n, puerto 5678) con sus cuatro workflows, y PostgreSQL (inventario_db, puerto 5432), además de los volúmenes que conservan los datos aunque los contenedores se apaguen. Dentro de Docker los contenedores se comunican por nombre de servicio: la aplicación llama a http://n8n:5678 y n8n se conecta a postgres. El archivo .env entrega a cada contenedor solo lo que necesita, y el comprobante PDF se descarga a la carpeta de descargas del navegador. Los tres contenedores arrancan solos al iniciar Docker Desktop.



14. Conclusiones

- La automatización con n8n permitió reemplazar un proceso manual por uno que valida, registra y reporta sin intervención humana, y que deja evidencia de cada operación.

- Delegar la consistencia del inventario a la base de datos, mediante una operación atómica y restricciones de integridad, resultó más confiable que comprobarla en pasos separados dentro del workflow.

- Separar la solución en workflows con una sola responsabilidad simplificó las pruebas y permitió que el manejo de errores rastreara exactamente dónde se originó cada fallo.

- La seguridad se integró desde el diseño (variables de entorno, control de acceso, validación y consultas parametrizadas) en lugar de agregarse al final.

- Repetir en la aplicación web las mismas reglas del WF1 creó una validación en dos capas: los errores se detectan antes de llegar a n8n y nadie puede saltarse las reglas.

- Las pruebas unitarias automatizadas permitieron modificar el código con confianza: cada cambio se verificó en segundos con 82 pruebas, y una de ellas ayudó a corregir un defecto antes de la entrega.

- Llevar la aplicación web a su propio contenedor Docker, con arranque automático, dejó el sistema completo disponible sin pasos manuales cada vez que se enciende la computadora.

14.1 Trabajo futuro

- Workflow de devolución que registre fecha_devolucion_real y restituya el stock.
- Notificaciones por correo a los estudiantes con préstamos vencidos.
- Autenticación de usuarios en la aplicación web, para registrar quién aprueba cada préstamo.
- Pruebas de integración automatizadas que levanten n8n y PostgreSQL en contenedores durante la ejecución de las pruebas.
- Envío automático del comprobante al celular del estudiante por WhatsApp, mediante la API de WhatsApp Business. Requiere una cuenta empresarial verificada y que Meta apruebe una plantilla del mensaje.

15. Referencias y recursos de apoyo

- n8n. Documentación oficial. https://docs.n8n.io
- PostgreSQL Global Development Group. PostgreSQL 16 Documentation. https://www.postgresql.org/docs/16/
- Docker Inc. Docker Compose Documentation. https://docs.docker.com/compose/
- Mermaid. Documentación de diagramas. https://mermaid.js.org
- Universidad Mariano Gálvez de Guatemala. Guía Completa del Proyecto Final – Análisis de Sistemas II.
- Spring. Spring Boot Reference Documentation. https://docs.spring.io/spring-boot/
- Thymeleaf. Tutorial: Using Thymeleaf. https://www.thymeleaf.org/documentation.html
- JUnit Team. JUnit User Guide. https://docs.junit.org/
- Mockito. Documentación oficial. https://site.mockito.org
- JaCoCo. Java Code Coverage Library. https://www.jacoco.org/jacoco/
- PlantUML. Guía de referencia del lenguaje. https://plantuml.com
- Apache Software Foundation. Apache PDFBox. https://pdfbox.apache.org
- Docker Inc. Dockerfile reference. https://docs.docker.com/reference/dockerfile/